Microsoft 365 → Google Workspace Migration Program & Wave Plan (Enterprise/150k+)

Implementation Solution Guide

Version 1.0.0 · 2026-07-06

Contents

Executive Summary — Microsoft 365 → Google Workspace Migration Program & Wave Plan 5

What you get . . . 5

What this delivers . . . 5

The deployment model . . . 6

Outcome . . . 6

1. Solution Overview — Microsoft 365 → Google Workspace Migration Program & Wave Plan 6

What this program is . . . 6

Why this matters . . . 6

The migration operating model . . . 6

The named program artifacts . . . 7

Workstream sequencing . . . 7

Wave planning in brief . . . 8

What this is not . . . 8

The deployment model . . . 8

2. Requirements Guide — Microsoft 365 → Google Workspace Migration Program & Wave Plan 8

2.1 A source-estate baseline . . . 9

2.2 A target & tooling design . . . 9

2.3 Identity & provisioning into Google . . . 9

2.4 A coexistence design (mail routing + calendar interop) . . . 9

2.5 A wave plan . . . 10

2.6 Per-wave cutover runbook and go/no-go gate . . . 10

2.7 A program cadence, RACI, and change control . . . 10

Checklist . . . 10

Global Requirements — Google Workspace & Gemini Solutions 11

1. Google Workspace / Cloud Identity tenant . . . 11

2. Source Microsoft 365 / Entra environment (for migration & coexistence) . . . 11

3. DNS control for the routed domains . . . 12

4. Identity & provisioning source of truth . . . 12

5. A pilot scope (OU / group) and change control . . . 12

6. Security, compliance & governance stakeholders . . . 12

3. Implementation Guide — Microsoft 365 → Google Workspace Migration Program & Wave Plan 13

Step 1 — Assess: baseline the source estate and stand up governance . . . 13

Step 2 — Design the target architecture and tooling strategy . . . 13

Step 3 — Design and stand up coexistence . . . 14
Step 4 — Build the wave plan . . . 14
Step 5 — Author the per-wave cutover runbook and go/no-go gate . . . 15
Step 6 — Run Wave 0 (the pilot) end to end . . . 15
Step 7 — Validate the pilot and fix the runbook, not just the wave . . . 15
Step 8 — Expand wave by wave . . . 16
Step 9 — Hypercare and decommission . . . 16
5. Testing Guide — Microsoft 365 → Google Workspace Migration Program & Wave Plan 16
Test 1 — Baseline and wave assignment are complete . . . 16
Test 2 — Readiness: identity and licensing provisioned . . . 17
Test 3 — Coexistence routes mail correctly . . . 17
Test 4 — Coexistence resolves calendar free/busy . . . 17
Test 5 — Mail-flow / DNS change applied under change control . . . 17
Test 6 — Data move completed for the wave . . . 17
Test 7 — Rollback is proven available . . . 17
Test 8 — Progress and coverage reported . . . 17
Pass/fail gate . . . 17
6. Troubleshooting Guide — Microsoft 365 → Google Workspace Migration Program & Wave Plan 18
A. Mail is lost or double-delivered during coexistence . . . 18
B. Cross-platform free/busy fails . . . 18
C. A wave fails its go/no-go gate . . . 18
D. The same problem recurs every wave . . . 19
E. A wave overwhelms migration throughput . . . 19
F. A user appears in two waves or none . . . 19
G. A cutover needs to be reversed . . . 19
H. Graph scripts fail to connect . . . 19
I. Diagnostics quick reference . . . 20
J. When to escalate . . . 20
7. Best Practices — Microsoft 365 → Google Workspace Migration Program & Wave Plan 20
8. Deliverables Summary — Microsoft 365 → Google Workspace Migration Program & Wave Plan 21
Documentation . . . 21
Named program artifacts . . . 22
PowerShell scripts . . . 22
The Migration-Program playbook (outline) . . . 23
What the buyer receives . . . 23

Outcome 24
Appendix A — PowerShell Scripts 25

Executive Summary — Microsoft 365 → Google Workspace Migration Program & Wave Plan

Moving 150k+ users from Microsoft 365 to Google Workspace is a **program, not a project** — and it fails without a disciplined wave plan, a coexistence design, and cutover governance. Most attempts stumble the same way: no wave plan, so a big-bang cutover meets 150k-scale problems a pilot would have caught; coexistence as an afterthought, so mail is lost or double-delivered while the estate is split; ungoverned cutovers, so waves are declared done before validation completes and fail with no rollback; and nothing proven first, so every wave re-does the same work. This flagship solution is the **end-to-end migration program** that removes those failure modes.

This is a complete, run-it-as-a-program package written for a senior migration program lead and architect. Every stage and artifact is named once and reused across every wave, so the same governed runbook executes from the pilot through the last business unit — the repeatable way.

What you get

• A **Solution Overview** of the migration operating model — discovery → design → coexistence → waves

→ cutover → hypercare — with workstream sequencing (identity first, coexistence before the first wave, mail/Drive/collaboration per wave, decommission last) and the named program artifacts.

• A **Requirements Guide** with verify / implement-if-missing for every prerequisite: a source-estate

baseline, a frozen target & tooling design, identity/provisioning into Google, a proven coexistence design, the wave plan, the per-wave runbook and gate, and the program cadence/RACI.

• An **Implementation Guide** of precise, ordered steps: baseline the estate, design the

target/tooling and coexistence, segment 150k users into sized waves, author the per-wave cutover runbook (readiness → mail-flow/DNS → data move → validation → rollback) and the go/no-go gate, prove it on a pilot wave, then expand wave by wave with hypercare and decommission.

• **Five PowerShell scripts** — source-estate discovery for wave planning, per-wave readiness

validation, a migration progress/coverage report, safe remediation, and a wave target-group builder — read-only against Microsoft Graph except the two write scripts, which default to `-WhatIf`.

• **Testing and Troubleshooting** guides with explicit per-wave gates, plus Best Practices for

executing waves without rework — and a **Migration-Program playbook outline** the program runs from.

The program **references, rather than duplicates,** the individual mail, Drive, and chat migration solutions: it sequences, segments, and governs them, keeping the tooling layer tool-agnostic so a tooling change touches the per-service solution, not the program.

What this delivers

A governed, wave-based migration: 150k+ users moved from Microsoft 365 to Google Workspace with coexistence managed during the transition, every cutover gated by an explicit go/no-go, and the cutover runbook proven on a pilot wave before any production business unit moves.

The deployment model

Assess, then Configure, then Pilot, then Validate, then Expand — with a per-wave go/no-go gate before every subsequent wave, and coexistence decommissioned only when the estate is fully migrated.

Outcome

The business moves off Microsoft 365 without breaking — waves execute the same proven runbook without architectural rework, coexistence keeps mail and calendars working while the estate is split, and the transition state is retired only after the final wave clears hypercare.

1. Solution Overview — Microsoft 365 → Google Workspace Migration Program & Wave Plan

What this program is

Moving an enterprise of 150k+ users from Microsoft 365 to Google Workspace is a **program**, not a project. It is a governed operating model — a discovery baseline, a wave plan, a tooling strategy, a coexistence design, per-wave cutover runbooks, and a hypercare period — run over months, across mail, Drive, and collaboration, with every wave gated before the next begins. This solution is that program. It orchestrates the individual data-movement workstreams (mail, Drive/OneDrive, Chat/Teams) rather than duplicating them: the per-service migration mechanics live in their own solutions (Exchange Online → Gmail, SharePoint/OneDrive → Google Drive, Teams → Google Chat/Meet); this program sequences them, segments the users, governs the cutovers, and holds the gates.

Why this matters

Enterprise migrations fail for predictable, non-technical reasons. There is no wave plan, so the organization attempts a big-bang cutover and discovers architectural problems at 150k scale that a pilot would have caught at 500. Coexistence is an afterthought, so mail is lost or double-delivered while half the estate is on each platform and free/busy lookups fail across the divide. Cutovers are ungoverned, so a wave is declared "done" while its DNS, mail-flow, and data-move validation is still outstanding, and the failure surfaces days later with no rollback. And because nothing is proven on a pilot first, every wave re-learns the same lessons and re-does the same work. A disciplined program fixes the cause: it segments the population deliberately, designs coexistence up front, governs each cutover with an explicit go/no-go gate, and proves the whole pattern on a pilot wave so later waves execute the same runbook without rework. Public enterprise-migration guidance is consistent on this — large moves are run in phases/waves for iterative testing, focused support, and continuous improvement, with core IT and early adopters first, then department-by-department rollout ([Google Workspace Migrate — plan your phases](https://support.google.com/workspacemigrate/answer/9275515), [Google enterprise migration & change management](https://workspace.google.com/solutions/migration/enterprise-migration/)).

The migration operating model

The program runs through six sequential stages. Each has an owner, an entry condition, and an exit gate (see the Implementation Guide). The stages are named and reused across every wave.

1. **Discovery** — baseline the source estate: user and mailbox counts, mailbox/Drive data volumes, shared mailboxes and resource calendars, distribution lists and groups, and the integrations (relay/apps/service accounts) that touch mail and files. This is what the wave plan is built from.

1. **Design** — decide the target Workspace architecture, the identity/provisioning model, the tooling strategy per service, and the coexistence design (how mail routes and free/busy works while the estate is split). Produced once, reused every wave.

1. **Coexistence** — stand up the transition state so migrated and not-yet-migrated users interoperate: directory sync, mail routing (dual/split delivery), and calendar interop. Coexistence is a **transitional** phase, deliberately time-boxed — never a permanent end state.

1. **Waves** — segment 150k users into sized, bounded waves and schedule them. Wave 0 is the pilot; subsequent waves expand by business unit/geography/data-weight until the estate is covered.

1. **Cutover** — execute each wave against its runbook: readiness → mail-flow/DNS → data move → validation → (rollback if the gate fails). Each wave passes a go/no-go gate before the next starts.

1. **Hypercare** — a heightened-support window after each wave (and after the final wave) where issues are triaged fast, coexistence is decommissioned in step, and the wave is formally closed.

The named program artifacts

The program is built from a fixed set of named artifacts you author once and reuse every wave. Using the exact names keeps the program legible across owners, waves, and months.

Artifact	Name
Source estate baseline	`GWMIGPROG Discovery - Source Estate - Baseline`
Target & tooling design	`GWMIGPROG Design - Target & Tooling - Program`
Coexistence design	`GWMIGPROG Coexistence - Mail & Calendar - Transition`
Wave plan	`GWMIGPROG Wave Plan - Segmentation - 150k`
Per-wave cutover runbook	`GWMIGPROG Runbook - Cutover - Per Wave`
Go/no-go gate	`GWMIGPROG Gate - Go/No-Go - Per Wave`
Program cadence & RACI	`GWMIGPROG Cadence & RACI - Program - All Stages`

Workstream sequencing

The program runs several workstreams in parallel, but they are ordered by dependency, not by convenience:

- **Identity & provisioning first.** Users and groups must exist in Google (via directory sync — e.g.

Google Cloud Directory Sync or SCIM) before any of their data can land or any coexistence routing can address them. Nothing migrates until identity is in place for that wave's population.

- **Coexistence before the first data wave.** Mail routing and calendar interop must be live before any user is cut over, so a migrated user and a not-yet-migrated user can still mail and schedule each other. Coexistence is stood up once and governs the whole transition.
- **Mail, Drive, and collaboration per wave.** Within a wave, data services migrate on a defined order (typically mail and calendar aligned to the mail cutover, Drive data staged ahead and finalized at cutover, chat/collaboration aligned to the user's move). Each service's mechanics are deferred to its own solution; the program schedules and gates them together per wave.
- **Decommission last.** Coexistence routing and the source tenant footprint are retired only after the final wave clears hypercare — never mid-program.

Wave planning in brief

Waves are sized from the discovery baseline. Wave count is driven by the number of users, the data volume, and the mix of data types being moved ([Google — plan your phases](https://support.google.com/workspacemigrate/answer/9275515)). The program spreads the most data-heavy users across waves so no single wave overwhelms the migration throughput, keeps each wave a coherent business unit/geography so support and comms are focused, and never moves a user between waves once the wave is confirmed (that risks duplicate data). Wave 0 is a 50–150-user pilot of core IT and early adopters that proves the full runbook end to end before any production business unit moves. The Implementation Guide gives the sizing method; `GWMIGPROG-Set-ScopeTargeting.ps1` builds each wave's target group.

What this is not

This program governs sequencing, segmentation, coexistence, and cutover — the orchestration layer. It does **not** re-document the internal steps of each per-service migration tool; those are deferred to the vendor's own docs and to the individual mail/Drive/Chat migration solutions. Where the program reads the source estate to plan and gate waves, the included scripts read **Microsoft 365 / Microsoft Entra** through Microsoft Graph, read-only; they move no data and change no mail flow.

The deployment model

> *Baseline the estate and design once, stand up coexistence, prove the full cutover runbook on a pilot wave, validate against the go/no-go gate, then expand wave by wave until the estate is covered — the repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand.*

2. Requirements Guide — Microsoft 365 → Google Workspace Migration Program & Wave Plan

Verify every requirement before running the program. Each entry gives Description, Why, Verify, and Implement-if-missing. Category-wide prerequisites (the Google tenant, the source M365/Entra environment, DNS control, identity source of truth, pilot scope, and governance stakeholders) are in

`global-requirements-google.md`; this guide adds what the migration program specifically needs.

2.1 A source-estate baseline

Description: A current inventory of the Microsoft 365 source — user and mailbox counts, mailbox and OneDrive/SharePoint data volumes, shared mailboxes and resource/room calendars, distribution lists and groups, and the mail-touching integrations (relay connectors, service accounts, apps) — captured as `GWMIGPROG Discovery - Source Estate - Baseline`. Why: The wave plan, the tooling strategy, and the coexistence design are all built from this baseline; without accurate counts and data volumes, waves are sized by guesswork and throughput estimates fail at scale. Verify: You can produce current user/mailbox counts and per-user data volumes for a representative sample; run `GWMIGPROG-Discovery.ps1`. Implement-if-missing: Run `GWMIGPROG-Discovery.ps1` against the source tenant to capture the baseline, then fill in the data-volume and integration details it cannot read, and declare it the baseline of record.

2.2 A target & tooling design

Description: A decision record — `GWMIGPROG Design - Target & Tooling - Program` — fixing the target Google Workspace architecture (edition/licensing, org-unit structure), the identity/provisioning model, and the migration tooling/pattern chosen per service (mail, Drive, chat/collaboration). Why: Waves must execute the *same* design; deciding tooling or target structure per wave guarantees rework and inconsistent outcomes across 150k users. Verify: The design names the provisioning model and the per-service tooling pattern, and it is agreed by the program's technical owners. Implement-if-missing: Hold the design step (Implementation Step 2), record the decisions, and freeze the design before Wave 0.

2.3 Identity & provisioning into Google

Description: Users and groups flow from the identity source of truth into Google before their data moves — via Google Cloud Directory Sync (GCDS) or SCIM-based provisioning — per the source of truth defined in the global requirements. Why: A user's mailbox, Drive, and chat cannot land, and coexistence cannot route to them, until the account exists in Google; identity is the first workstream and every wave depends on it. Verify: A sample wave's users and groups appear in the Google Admin console before any data move; run `GWMIGPROG-Validation.ps1` for that wave. Implement-if-missing: Stand up GCDS/SCIM provisioning against the source of truth and confirm a test population provisions correctly before scheduling Wave 0.

2.4 A coexistence design (mail routing + calendar interop)

Description: A designed transition state — `GWMIGPROG Coexistence - Mail & Calendar - Transition` — defining how inbound mail routes while the estate is split (dual or split delivery), how migrated and not-yet-migrated users still exchange mail, and how calendar free/busy works across the two platforms. Why: During the months the estate is split, mail must reach the right platform for each user and free/busy must resolve across the divide; without a coexistence design mail is lost or double-delivered and cross-platform scheduling fails. Split delivery routes inbound mail to Google or Microsoft 365 per recipient/mailbox rules; dual delivery delivers to a primary and forwards to a secondary ([Google — dual delivery](https://support.google.com/a/answer/9228551), [Google — split delivery](https://support.google.com/a/answer/12971016)). Verify: The design specifies the routing pattern and the free/busy interop approach, and a test message and a test cross-platform meeting resolve correctly in a lab. Implement-if-missing: Design coexistence in Step 3, stand up routing on a test

domain/host, and prove it before the first production wave. Coexistence is transitional — set its intended end.

2.5 A wave plan

Description: A segmentation of the 150k-user population into sized, bounded, scheduled waves — named `GWMIGPROG Wave Plan - Segmentation - 150k` — with Wave 0 as the pilot. Why: Wave count and size are driven by user numbers, data volume, and data types ([Google — plan your phases](https://support.google.com/workspacemigrate/answer/9275515)); a plan is what turns a 150k population into executable, gated units and keeps data-heavy users from swamping any single wave. Verify: Every in-scope user is assigned to exactly one wave, waves are sized against migration throughput, and the pilot wave is identified. Implement-if-missing: Build the wave plan from the discovery baseline using the sizing method in the Implementation Guide; build each wave's target group with `GWMIGPROG-Set-ScopeTargeting.ps1`.

2.6 Per-wave cutover runbook and go/no-go gate

Description: A repeatable per-wave runbook — `GWMIGPROG Runbook - Cutover - Per Wave` — with the fixed sequence (readiness → mail-flow/DNS → data move → validation → rollback) and an explicit `GWMIGPROG Gate - Go/No-Go - Per Wave` that must pass before the next wave starts. Why: Every wave executes the same governed cutover; an ungoverned cutover is declared done before its validation completes, and a failed cutover with no rollback becomes an incident. Verify: The runbook exists with all five stages and a documented rollback, and the gate criteria are defined and owned. Implement-if-missing: Author the runbook and gate in Steps 5–6, prove them on Wave 0, and reuse them unchanged for every subsequent wave.

2.7 A program cadence, RACI, and change control

Description: A recurring program forum with a RACI — `GWMIGPROG Cadence & RACI - Program - All Stages` — that owns the wave schedule, chairs each go/no-go, and operates under the change-control process for production mail-flow, DNS, and identity changes. Why: A months-long program across mail, DNS, identity, and multiple business units needs named owners and a decision forum, or waves stall and high-impact changes go in unreviewed. Verify: A recurring forum exists with the right owners, and every wave's cutover stages have a named Responsible and Accountable owner. Implement-if-missing: Stand up the cadence and RACI in Step 1 and align it to the change-control process from the global requirements.

Checklist

- Source-estate baseline captured — `GWMIGPROG-Discovery.ps1` → `GWMIGPROG Discovery - Source Estate - Baseline`
- Target & tooling design frozen — `GWMIGPROG Design - Target & Tooling - Program`
- Identity/provisioning into Google live — `GWMIGPROG-Validation.ps1`
- Coexistence design proven — `GWMIGPROG Coexistence - Mail & Calendar - Transition`
- Wave plan built (every user in one wave) — `GWMIGPROG Wave Plan - Segmentation - 150k`

• Per-wave runbook + go/no-go gate authored — `GWMIGPROG Runbook - Cutover - Per Wave`

• Program cadence + RACI + change control — `GWMIGPROG Cadence & RACI - Program - All Stages`

Global Requirements — Google Workspace & Gemini Solutions

These requirements apply to **every** solution in the **Google** category. They are the common foundation each Google Workspace / Gemini solution assumes is already in place (or stands up as part of its own Requirements section). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Google solution).*** *The **Google Admin console** (admin.google.com), **Google Cloud / Cloud Identity**, and the **Microsoft / Entra** sides are authored at exact, current click-by-click detail from the vendors' public documentation. **Gemini Enterprise** capabilities that are in preview or rolling out are **cited or flagged "verify in your tenant"** — never invented. **Third-party migration tools** (e.g. for mail/Drive/Chat data movement) and **third-party enterprise apps** (ServiceNow, SAP, Databricks, etc.) are integrated at the pattern/architectural level; their internal console steps are deferred to the vendor's own docs.*

1. Google Workspace / Cloud Identity tenant

• **Description:** A provisioned Google Workspace (or Cloud Identity) tenant with a **super-admin** account for configuration, and the appropriate Workspace edition/licensing for the capability in scope (e.g. Enterprise Plus for Vault/CAA/DLP, Gemini/AI add-on where required).

• **Why required:** All configuration, policy, and API work is performed against this tenant.

• **How to verify:** Sign in to **admin.google.com** as a super-admin; confirm edition and license counts under **Billing / Subscriptions**.

• **How to implement if missing:** Provision the tenant and licenses through Google or an authorized reseller before starting.

2. Source Microsoft 365 / Entra environment (for migration & coexistence)

• **Description:** For migration/coexistence/identity solutions, administrative access to the source **Microsoft 365** tenant and **Microsoft Entra ID** (Global Administrator or the least-privilege roles the solution names).

• **Why required:** Migration, directory sync, SSO federation, and coexistence read from and change the Microsoft side.

• **How to verify:** Confirm access to the M365 admin center and the Entra admin center with the required role.

• **How to implement if missing:** Obtain the role from the M365/Entra tenant owner.

3. DNS control for the routed domains

• **Description:** Ability to create/modify **DNS records** (MX, SPF, DKIM, DMARC, TXT, CNAME) for the domains being migrated, coexisting, or verified.

• **Why required:** Domain verification, mail routing/coexistence, and authentication all depend on DNS changes.

• **How to verify:** Confirm access to the authoritative DNS zone for each domain.

• **How to implement if missing:** Coordinate with whoever controls DNS; stage changes under change control (mail-flow changes are high-impact).

4. Identity & provisioning source of truth

• **Description:** A defined **identity source of truth** (on-prem AD, Entra ID, or an HR/IGA system) and the sync mechanism (e.g. **Google Cloud Directory Sync (GCDS)** or SCIM-based provisioning).

• **Why required:** Consistent users/groups across ecosystems underpin SSO, licensing, and access.

• **How to verify:** Confirm which directory is authoritative and how objects flow into Google.

• **How to implement if missing:** Define the source of truth and provisioning model before configuring SSO or migrating.

5. A pilot scope (OU / group) and change control

• **Description:** A **pilot organizational unit or group** to validate on before fleet-wide rollout, and a **change-control** process for production changes (especially mail flow, DNS, identity, and DLP).

• **Why required:** Every solution follows an **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model; the pilot proves the design and change control protects production.

• **How to verify:** Confirm a named pilot OU/group exists and a change window/approval path is available.

• **How to implement if missing:** Create a pilot OU/group and agree the change-control process with the owners before making production changes.

6. Security, compliance & governance stakeholders

• **Description:** Engagement with the **security (CISO), legal/eDiscovery, and data-governance** stakeholders whose policies the configuration must satisfy (DLP, retention, Vault/Purview, and any regulatory regime such as ITAR/DFARS/Assured Controls where applicable).

• **Why required:** Data governance, eDiscovery, DLP, and regulated-workload settings must align to enterprise policy and legal requirements, not just technical defaults.

• **How to verify:** Confirm the applicable policies and named stakeholders/approvers.

• **How to implement if missing:** Identify the owners and obtain the policy/compliance requirements before configuring governance controls.

*The deployment model across every Google solution is the same: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes.*

3. Implementation Guide — Microsoft 365 → Google Workspace Migration Program & Wave Plan

Follow these steps in order. This guide runs the migration program end to end: baseline the estate, design the target/tooling and coexistence, build the wave plan, prove the full cutover runbook on a pilot wave, then expand wave by wave until the 150k estate is covered. Artifact names are fixed — create them with the exact names shown and reuse them for every wave. The per-service data-movement mechanics (mail, Drive, chat) are executed from their own solutions; this guide sequences and gates them.

> *Model: baseline and design once, stand up coexistence, prove the cutover on a pilot wave, validate against the gate, then expand. The repeatable way: Assess, then Configure, then Pilot, then Validate, then Expand.*

Step 1 — Assess: baseline the source estate and stand up governance

1. Run `GWMIGPROG-Discovery.ps1` to capture the source baseline: user/mailbox counts, licensed users,

shared mailboxes and resource calendars, distribution lists/groups, and mail-touching signals. Record it as `GWMIGPROG Discovery - Source Estate - Baseline`, then add the data-volume and integration details the script cannot read (per-user mailbox/OneDrive sizes, relay connectors, service accounts, apps that send mail).

1. Stand up `GWMIGPROG Cadence & RACI - Program - All Stages`: a recurring program forum with named

Responsible/Accountable owners for each cutover stage, chairing every go/no-go, operating under the change-control process from the global requirements.

1. Record gaps against the Requirements checklist. Do not proceed past a missing baseline or missing identity source of truth — resolve those first.

Step 2 — Design the target architecture and tooling strategy

1. Fix the target Google Workspace architecture: edition/licensing for the estate, org-unit structure,

and group model. Record it in `GWMIGPROG Design - Target & Tooling - Program`.

1. Choose the identity/provisioning model (GCDS or SCIM) against the identity source of truth.

2. Choose the migration tooling/pattern **per service** and write it into the design:

• **Mail & calendar** — the tool/pattern that moves mailboxes, calendars, and contacts and the

order relative to the mail cutover (mechanics deferred to the Exchange Online → Gmail solution).

• **Drive** — the tool/pattern that moves OneDrive/SharePoint content to Google Drive/shared drives with permissions preserved, staged ahead of cutover and finalized at cutover (mechanics deferred to the SharePoint/OneDrive → Google Drive solution).

• **Chat / collaboration** — the tool/pattern for Teams → Google Chat/Meet aligned to the user's move (mechanics deferred to the Teams → Chat/Meet solution). Author the tooling **tool-agnostically** here — this program governs *which pattern and when*, not a specific vendor's console steps.

1. Freeze the design before Wave 0. Every wave executes this same design.

Step 3 — Design and stand up coexistence

1. Design the transition state as `GWMIGPROG Coexistence - Mail & Calendar - Transition`:

• **Mail routing.** Decide **split delivery** (inbound mail routed to Google or Microsoft 365 per recipient/mailbox rules — the usual pattern for a per-wave move where MX points at Google and not-yet-migrated users are routed on to Microsoft 365) or **dual delivery** (deliver to a primary and forward to a secondary), per [Google — dual delivery](https://support.google.com/a/answer/9228551) and [Google — split delivery](https://support.google.com/a/answer/12971016). Define how the routing rule keys off each wave's migrated population so mail follows users as they move.

• **Calendar interop.** Define how free/busy resolves across the two platforms during the split so cross-platform scheduling works.

1. Stand up the routing on a test domain/host and prove it: a test message reaches the correct platform for a migrated and a not-yet-migrated user, and a cross-platform meeting resolves free/busy.

1. Set coexistence's intended **end** — it is transitional, decommissioned as waves complete, never a permanent state.

Step 4 — Build the wave plan

1. Segment the 150k population into `GWMIGPROG Wave Plan - Segmentation - 150k`. Size waves from the discovery baseline, driven by user count, data volume, and data types ([Google — plan your phases](https://support.google.com/workspacemigrate/answer/9275515)).

1. Apply the sizing rules:

• **Wave 0 = pilot** — 50–150 users of core IT and early adopters that exercises the full runbook end to end before any production business unit moves.

• **Coherent business units.** Keep each wave a single business unit/geography so comms, support, and coexistence routing are focused.

• **Spread data-heavy users.** Distribute the largest mailboxes/Drives across waves so no single wave exceeds migration throughput.

• **Essential data at go-live, archives later** where the tooling allows deferring low-value historical data past cutover.

• **One wave per user, fixed.** Assign every user to exactly one wave and do not move users between waves once confirmed (that risks duplicate data).

1. Build each wave's target group with `GWMIGPROG-Set-ScopeTargeting.ps1` (runs with `-WhatIf` by default).

Step 5 — Author the per-wave cutover runbook and go/no-go gate

1. Author `GWMIGPROG Runbook - Cutover - Per Wave` with the fixed five-stage sequence:

2. **Readiness** — the wave's users/groups are provisioned in Google, licenses assigned, Drive pre-stage complete, comms/training delivered, and `GWMIGPROG-Validation.ps1` reports the wave ready.

1. **Mail-flow / DNS** — apply the coexistence routing change for this wave under change control so mail follows the migrated users; stage any DNS/MX change with a documented back-out.

1. **Data move** — execute the per-service data move for the wave (mail/calendar, Drive finalize, chat) via the individual solutions, in the designed order.

1. **Validation** — run the per-wave validation gates (Testing Guide) and `GWMIGPROG-Report.ps1` to confirm coverage.

1. **Rollback** — the documented back-out (revert routing/DNS to the prior platform, re-point the wave) used only if the gate fails.

1. Define `GWMIGPROG Gate - Go/No-Go - Per Wave`: the wave passes only when readiness held, mail-flow is correct, the data move completed for the wave's users, validation passed, and hypercare owners are ready. The next wave does not start until this gate passes.

Step 6 — Run Wave 0 (the pilot) end to end

1. Build the Wave 0 group with `GWMIGPROG-Set-ScopeTargeting.ps1 -WhatIf:$false`.

2. Execute the full runbook on Wave 0: readiness → mail-flow/DNS → data move → validation.

3. Run `GWMIGPROG-Validation.ps1` for the wave and `GWMIGPROG-Report.ps1` for progress/coverage.

4. Hold the Wave 0 `GWMIGPROG Gate - Go/No-Go - Per Wave` at the program cadence. The pilot passes only when the whole runbook ran end to end and validation is clean.

Step 7 — Validate the pilot and fix the runbook, not just the wave

1. Complete the Testing Guide for Wave 0. Treat every issue as a **runbook** defect: fix the runbook, the coexistence design, or the tooling pattern so later waves inherit the fix.

1. Remediate any data/coverage gaps with `GWMIGPROG-Remediation.ps1` (runs with `-WhatIf` by default) before re-running validation.

Step 8 — Expand wave by wave

1. For each subsequent wave, re-run the same runbook: build the wave group (`GWMIGPROG-Set-ScopeTargeting.ps1`), execute readiness → mail-flow/DNS → data move → validation, run `GWMIGPROG-Validation.ps1` and `GWMIGPROG-Report.ps1`, and pass the go/no-go gate before the next wave.

1. Keep coexistence routing current as the migrated population grows; the routing rule must always send each user's mail to their current platform.

Step 9 — Hypercare and decommission

1. After each wave, run a heightened-support hypercare window: triage issues fast, confirm the wave's users are fully functional, and formally close the wave at the cadence.

1. After the **final** wave clears hypercare, decommission coexistence routing, retire the transition DNS/MX configuration, and stand down the source-tenant footprint per the design's end-state — never mid-program.

Order of operations

```
Assess/baseline -> Design (target+tooling) -> Coexistence -> Wave plan -> Runbook+gate -> Wave 0 pilot -> Validate/fix runbook -> Expand wave by wave -> Hypercare + decommission
```

5. Testing Guide — Microsoft 365 → Google Workspace Migration Program & Wave Plan

Run these per-wave validation gates for **every** wave — first for Wave 0 (the pilot), then for each expansion wave — before that wave's go/no-go decision. Each test confirms one part of the cutover runbook worked for the wave's population. A wave does not pass until its gate criteria are met.

Test 1 — Baseline and wave assignment are complete

Do: Confirm every user in the wave is assigned to exactly one wave in `GWMIGPROG Wave Plan - Segmentation - 150k` and appears in the wave's target group; run `GWMIGPROG-Set-ScopeTargeting.ps1` (preview) to confirm membership. Expected: The wave's members match the plan; no user is in two waves and none is unassigned.

Test 2 — Readiness: identity and licensing provisioned

Do: Run `GWMIGPROG-Validation.ps1` for the wave. Expected: The wave's users/groups are provisioned into Google and licensed; readiness signals report present with no gaps.

Test 3 — Coexistence routes mail correctly

Do: Send a test message to a migrated wave user and to a not-yet-migrated user; send one from each back out. Expected: Each message reaches the correct platform per the split/dual-delivery design — no loss, no double delivery — matching `GWMIGPROG Coexistence - Mail & Calendar - Transition`.

Test 4 — Coexistence resolves calendar free/busy

Do: Create a cross-platform meeting between a migrated user and a not-yet-migrated user. Expected: Free/busy resolves both ways and the invite is delivered on both platforms.

Test 5 — Mail-flow / DNS change applied under change control

Do: Confirm the wave's routing/DNS change was applied with an approved change record and a documented back-out. Expected: The change is recorded, reversible, and mail follows the wave's migrated users.

Test 6 — Data move completed for the wave

Do: For a sample of wave users, confirm mail/calendar, Drive content (permissions preserved), and chat/collaboration moved per the design; run `GWMIGPROG-Report.ps1`. Expected: The report shows the wave's users migrated and covered across the in-scope services; sampled users can access their moved data in Google Workspace.

Test 7 — Rollback is proven available

Do: Review the wave's rollback step and confirm the back-out (revert routing/DNS, re-point the wave) is documented and executable. Expected: A clear, tested rollback exists for the wave; it is invoked only if the gate fails.

Test 8 — Progress and coverage reported

Do: Run `GWMIGPROG-Report.ps1` for the program. Expected: Cumulative migration progress/coverage (waves complete, users migrated vs total, coexistence scope) renders without gaps for the wave.

Pass/fail gate

Gate	Requirement
Wave 0 (pilot) pass	Tests 1-8 pass; the full runbook ran end to end and every runbook defect is fixed in the runbook/design before expanding

Gate	Requirement
Expansion wave pass	Tests 1-8 pass for the wave; the go/no-go gate `GWMIGPROG Gate - Go/No-Go - Per Wave` is cleared before the next wave starts
Program decommission	All waves passed and cleared hypercare before coexistence routing and the source-tenant footprint are retired

6. Troubleshooting Guide — Microsoft 365 → Google Workspace Migration Program & Wave Plan

Use `GWMIGPROG-Discovery.ps1` for the source baseline, `GWMIGPROG-Validation.ps1` for per-wave readiness, `GWMIGPROG-Report.ps1` for progress/coverage, and `GWMIGPROG-Remediation.ps1` (runs with `-WhatIf` by default) for safe fixes to wave-readiness data gaps. Per-service data-movement failures (a specific mailbox, Drive item, or chat) are diagnosed in the individual mail/Drive/chat migration solutions and the tool's own docs; this guide covers the **program**: waves, coexistence, cutover, and gates.

A. Mail is lost or double-delivered during coexistence

Symptom	Cause	Remediation
Migrated user's mail still lands in Microsoft 365	Routing rule not updated after the wave	Update `GWMIGPROG Coexistence - Mail & Calendar - Transition` routing so the wave's users route to Google; re-test with Testing Guide Test 3
A message is delivered to both platforms	Dual delivery left on for users who have cut over	Switch those users to split-delivery routing (single target = Google); confirm no forwarding loop remains
Not-yet-migrated user's mail bounces	Routing has no on-ward path to Microsoft 365	Confirm the secondary host/route to Microsoft 365 exists per the coexistence design ([Google — split delivery](https://support.google.com/a/answer/12971016))

B. Cross-platform free/busy fails

- A migrated and a not-yet-migrated user cannot see each other's availability. Confirm the calendar interop in `GWMIGPROG Coexistence - Mail & Calendar - Transition` is live for both directions and re-run Testing Guide Test 4 before continuing the wave.

C. A wave fails its go/no-go gate

Symptom	Cause	Remediation
Readiness fails	Users not provisioned/licensed in Google	Fix identity/provisioning (GCDS/SCIM) for the wave; re-run `GWMIGPROG-Validation.ps1`

Symptom	Cause	Remediation
Validation incomplete	Data move not finished for the wave's users	Do not pass the gate; complete the per-service move, then re-run `GWMIGPROG-Report.ps1`
Gate ambiguous	Criteria not defined/owned	Complete `GWMIGPROG Gate - Go/No-Go - Per Wave` criteria and name the Accountable owner before deciding

D. The same problem recurs every wave

• A defect fixed only for one wave will reappear. Fix the **runbook**, coexistence design, or tooling

pattern — not just the wave — so later waves inherit the fix (Implementation Step 7). Recurring issues are almost always an un-fixed runbook, not bad luck.

E. A wave overwhelms migration throughput

• A wave runs long or stalls. The wave is likely too large or too data-heavy. Re-balance

`GWMIGPROG Wave Plan - Segmentation - 150k` to spread data-heavy users across waves and pre-stage Drive content ahead of cutover ([Google — plan your phases](https://support.google.com/workspacemigrate/answer/9275515)), then rebuild the wave group with `GWMIGPROG-Set-ScopeTargeting.ps1`.

F. A user appears in two waves or none

• Duplicate or missing wave assignment causes duplicate data or a stranded user. Enforce one-user-one-

wave in the wave plan; run `GWMIGPROG-Set-ScopeTargeting.ps1` (preview) and `GWMIGPROG-Validation.ps1` to reconcile membership before cutover. Never move a confirmed user between waves.

G. A cutover needs to be reversed

• The wave failed validation after the routing/DNS change. Invoke the runbook's rollback: revert the

coexistence routing/DNS change to the prior platform under change control and re-point the wave. Rollback is only used when the gate fails — confirm it was tested (Testing Guide Test 7).

H. Graph scripts fail to connect

• `GWMIGPROG-Discovery.ps1`, `GWMIGPROG-Validation.ps1`, `GWMIGPROG-Report.ps1`, and

`GWMIGPROG-Set-ScopeTargeting.ps1` use Microsoft Graph against the source M365/Entra tenant. If connection fails, confirm the Microsoft Graph PowerShell SDK is installed and the signed-in account holds the read scopes the script requests (consent on first run). Re-run after consent.

I. Diagnostics quick reference

```
.\GWMIGPROG-Discovery.ps1                                # source-estate baseline for wave planning
.\GWMIGPROG-Set-ScopeTargeting.ps1 -GroupName "..." -MembersCsv .\wave.csv   # -WhatIf by default
.\GWMIGPROG-Validation.ps1 -Wave "Wave 1" -MembersCsv .\wave.csv             # per-wave readiness
.\GWMIGPROG-Report.ps1 -WavePlanCsv .\wave-plan.csv    # progress/coverage
.\GWMIGPROG-Remediation.ps1 -MembersCsv .\wave.csv     # -WhatIf by default; add -WhatIf:$false to act
```

J. When to escalate

If a wave's users are provisioned and licensed, coexistence routing is correct, the data move completed, and validation still reports gaps after remediation, capture the `GWMIGPROG-Validation.ps1` and `GWMIGPROG-Report.ps1` output and review the wave with the identity, mail-flow, and per-service migration owners at the program cadence before passing the go/no-go gate.

7. Best Practices — Microsoft 365 → Google Workspace Migration Program & Wave Plan

Field-proven practices for executing 150k users of migration wave by wave **without rework**.

- **Treat it as a program, not a project.** Baseline, design, coexistence, waves, cutover, and

hypercare are stages with owners and gates — run them as one governed operating model, not a series of ad-hoc moves.

- **Design once, execute many.** Freeze `GWMIGPROG Design - Target & Tooling - Program` and the

coexistence design before Wave 0. Deciding target structure or tooling per wave is the single biggest source of rework at scale.

- **Prove the whole runbook on a pilot wave.** Wave 0 (core IT + early adopters) exercises readiness →

mail-flow/DNS → data move → validation → rollback end to end. It is cheaper to find a defect in 100 users than in a business unit of 10,000.

- **Fix the runbook, not just the wave.** Every issue found in a wave is a defect in the runbook,

coexistence design, or tooling pattern — fix it there so all later waves inherit the fix. This is what stops each wave re-learning the same lesson.

- **Identity before data, always.** Provision each wave's users/groups into Google (GCDS/SCIM) before

any data moves or any routing addresses them; a data move to a non-existent account fails loudly at scale.

- **Stand up coexistence before the first wave, and time-box it.** Mail routing and calendar interop

must be live before anyone cuts over so migrated and not-yet-migrated users still interoperate — and coexistence is transitional, decommissioned as waves complete, never left running indefinitely ([Google — dual delivery](https://support.google.com/a/answer/9228551), [Google — split delivery](https://support.google.com/a/answer/12971016)).

- **Keep the routing rule following users.** As each wave completes, update coexistence routing so

every user's mail always reaches their current platform; a stale rule silently mis-delivers.

• **Spread data-heavy users across waves.** Distribute the largest mailboxes/Drives so no wave exceeds

migration throughput; pre-stage Drive content ahead of cutover and finalize at cutover ([Google — plan your phases](https://support.google.com/workspacemigrate/answer/9275515)).

• **One user, one wave, fixed.** Assign every user to exactly one wave and never move a user between

waves once confirmed — moving users mid-flight risks duplicate data.

• **Gate every cutover.** No wave starts until the previous wave passes

`GWMIGPROG Gate - Go/No-Go - Per Wave`; a wave declared done before its validation completes is an incident waiting to surface.

• **Always have a rollback.** Every wave's routing/DNS change is applied under change control with a

documented, tested back-out; a high-impact mail-flow change without a rollback is not ready to run.

• **Change management runs in tandem with the technical plan.** Comms and role-based training are part

of each wave's readiness, set up alongside the migration plan from the start — not bolted on at cutover ([Google — enterprise migration & change management](https://workspace.google.com/solutions/migration/enterprise-migration/)).

• **Defer tool internals to the per-service solutions.** This program governs *which pattern and when*;

the mail, Drive, and chat migration mechanics live in their own solutions and vendor docs — keep the program tool-agnostic so a tooling change touches the per-service layer, not the program.

• **Report progress cumulatively.** Track waves complete, users migrated vs total, and coexistence

scope every cadence with `GWMIGPROG-Report.ps1`; a stalled coverage trend is an early warning.

8. Deliverables Summary — Microsoft 365 → Google Workspace Migration Program & Wave Plan

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The migration operating model (discovery → design → coexistence → waves → cutover → hypercare), workstream sequencing, and the named program artifacts (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Google)	Category-wide prerequisites, published once

Document	Purpose
3. Implementation Guide	Exact, ordered steps: baseline, design, coexistence, wave plan, per-wave cutover runbooks, go/no-go gates, hypercare
5. Testing Guide	Per-wave validation gates with pass/fail criteria
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Executing waves without rework

Named program artifacts

Artifact	Name
Source estate baseline	`GWMIGPROG Discovery - Source Estate - Baseline`
Target & tooling design	`GWMIGPROG Design - Target & Tooling - Program`
Coexistence design	`GWMIGPROG Coexistence - Mail & Calendar - Transition`
Wave plan	`GWMIGPROG Wave Plan - Segmentation - 150k`
Per-wave cutover runbook	`GWMIGPROG Runbook - Cutover - Per Wave`
Go/no-go gate	`GWMIGPROG Gate - Go/No-Go - Per Wave`
Program cadence & RACI	`GWMIGPROG Cadence & RACI - Program - All Stages`

PowerShell scripts

Script	What it does
`GWMIGPROG-Discovery.ps1`	Baselines the Microsoft 365 source estate for wave planning: user/mailbox counts, licensed users, shared mailboxes, resource/room mailboxes, and groups (Microsoft Graph, read-only)
`GWMIGPROG-Validation.ps1`	Confirms per-wave readiness: the wave's users are provisioned, licensed, and coexistence/target signals are in place (read-only)
`GWMIGPROG-Report.ps1`	Renders migration progress/coverage: waves complete, users migrated vs total, and per-wave coverage (Microsoft Graph, read-only)
`GWMIGPROG-Remediation.ps1`	Safe remediation of wave-readiness data gaps; runs with `-WhatIf` by default

Script	What it does
`GWMIGPROG-Set-ScopeTargeting.ps1`	Builds a wave's target group in Microsoft Entra from a members CSV; runs with `-WhatIf` by default

The Migration-Program playbook (outline)

A single reusable playbook the program runs from, assembled from the artifacts above:

1. **Program charter** — scope (150k users, in-scope services), the operating model's six stages,

and `GWMIGPROG Cadence & RACI - Program - All Stages`.

1. **Discovery baseline** — `GWMIGPROG Discovery - Source Estate - Baseline`: counts, data volumes,

shared/resource mailboxes, groups, and mail-touching integrations.

1. **Target & tooling design** — `GWMIGPROG Design - Target & Tooling - Program`: target architecture,

identity/provisioning model, and the per-service tooling pattern (mail, Drive, chat), tool-agnostic.

1. **Coexistence design** — `GWMIGPROG Coexistence - Mail & Calendar - Transition`: mail routing

(split/dual delivery), calendar interop, and the intended decommission point.

1. **Wave plan** — `GWMIGPROG Wave Plan - Segmentation - 150k`: Wave 0 pilot, coherent business-unit

waves, data-heavy users spread, one user per wave.

1. **Per-wave cutover runbook** — `GWMIGPROG Runbook - Cutover - Per Wave`: readiness → mail-flow/DNS →

data move → validation → rollback.

1. **Go/no-go gate** — `GWMIGPROG Gate - Go/No-Go - Per Wave`: the criteria each wave clears before the

next starts.

1. **Hypercare & decommission** — per-wave hypercare, wave closure, and end-of-program coexistence and

source-tenant decommission.

What the buyer receives

- This implementation guide as a Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable migration program following the model: Assess, then Configure, then Pilot, then

Validate, then Expand.

Outcome

150k+ users moved from Microsoft 365 to Google Workspace wave by wave — coexistence managed, cutovers governed by go/no-go gates, and the cutover runbook proven on a pilot wave — so waves execute without architectural rework and the transition state is decommissioned only when the estate is fully migrated.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

GWMIGPROG-Discovery.ps1

Purpose: Captures the source-estate baseline the wave plan is built from: total and licensed user counts, shared mailboxes, resource/room mailboxes, and group/distribution-list counts read from the Microsoft 365 / Entra source through Microsoft Graph. Read-only — it supports the Assess step and moves no data and changes no mail flow. Data volumes and mail-touching integrations the API cannot read are recorded by hand into the baseline artifact. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWMIGPROG-Discovery.ps1 Optional: -CsvPath .\gwmigprog-baseline.csv to export the summary. Requires the Microsoft Graph PowerShell SDK. On first run, consent to the read scopes: User.Read.All, Organization.Read.All, Group.Read.All. EXPECTED RESULTS A baseline summary: total users, licensed users, shared mailboxes, resource/room mailboxes, and group count, with a PASS/GAP line per signal you record against the Requirements checklist as GWMIGPROG Discovery - Source Estate - Baseline. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\GWMIGPROG-Discovery.ps1 Optional: -CsvPath .\gwmigprog-baseline.csv to export the summary. Requires the Microsoft Graph PowerShell SDK. On first run, consent to the read scopes: User.Read.All, Organization.Read.All, Group.Read.All. EXPECTED RESULTS A baseline summary: total users, licensed users, shared mailboxes, resource/room mailboxes, and group count, with a PASS/GAP line per signal you record against the Requirements checklist as GWMIGPROG Discovery - Source Estate - Baseline. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$CsvPath)

GWMIGPROG-Remediation.ps1

Purpose: Cleans common data-quality gaps that block a wave from passing readiness, working only on a local wave members CSV — never on tenant data, mail flow, or money. It flags duplicate UPNs, blank UPN rows, and (when a source is reachable) UPNs that do not resolve in the Microsoft 365 source, and when applied writes a normalized, de-duplicated members CSV you can re-use. Runs with -WhatIf by default, so it only reports unless you explicitly turn -WhatIf off. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\GWMIGPROG-Remediation.ps1 `-MembersCsv .\wave1-users.csv` `Apply: ...\GWMIGPROG-Remediation.ps1 -MembersCsv .\wave1-users.csv` -OutputPath .\wave1-users.clean.csv -Resolve -WhatIf:$false -Resolve additionally checks each UPN against the source tenant (needs the Microsoft Graph SDK and User.Read.All). Without -Resolve, only local de-duplication and blank checks run. EXPECTED RESULTS In preview: a list of duplicate/blank (and, with -Resolve, unresolved) UPNs and what would change. When applied: a cleaned members CSV at -OutputPath (duplicates collapsed, blanks dropped), leaving the original untouched. .NOTES White-label — the repeatable way. Local-file only unless -Resolve is used; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([Parameter(Mandatory)][string]$MembersCsv, [string]$OutputPath, [switch]$Resolve)

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\GWMIGPROG-Remediation.ps1 `-MembersCsv .\wave1-users.csv` `Apply: ...\GWMIGPROG-Remediation.ps1 -MembersCsv .\wave1-users.csv` -OutputPath .\wave1-users.clean.csv -Resolve -WhatIf:$false -Resolve additionally checks each UPN against the

source tenant (needs the Microsoft Graph SDK and User.Read.All). Without -Resolve, only local de-duplication and blank checks run. EXPECTED RESULTS In preview: a list of duplicate/blank (and, with -Resolve, unresolved) UPNs and what would change. When applied: a cleaned members CSV at -OutputPath (duplicates collapsed, blanks dropped), leaving the original untouched. .NOTES White-label — the repeatable way. Local-file only unless -Resolve is used; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'Medium')] param([Parameter(Mandatory)][string]$MembersCsv, [string]$OutputPath, [switch]$Resolve)

GWMIGPROG-Report.ps1

Purpose: Produces the standing program report: waves complete vs planned, users migrated vs the total source population, and per-wave coverage. It reads the wave plan from a CSV (one row per wave with planned/migrated counts and status) and cross-checks the total population against the Microsoft 365 source through Microsoft Graph. Read-only against Graph — it modifies nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWMIGPROG-Report.ps1 ` -WavePlanCsv .\gwmigprog-wave-plan.csv -CsvPath .\gwmigprog-progress.csv The wave-plan CSV should include columns: Wave, PlannedUsers, MigratedUsers, Status (Planned/InProgress/Complete). Requires the Microsoft Graph PowerShell SDK and User.Read.All (used only to read the total source-user count for the coverage denominator). EXPECTED RESULTS A per-wave progress table, program totals (waves complete, users migrated vs source total, overall coverage %), and an optional CSV export. No source data is modified. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$WavePlanCsv, [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\GWMIGPROG-Report.ps1 ` -WavePlanCsv .\gwmigprog-wave-plan.csv -CsvPath .\gwmigprog-progress.csv The wave-plan CSV should include columns: Wave, PlannedUsers, MigratedUsers, Status (Planned/InProgress/Complete). Requires the Microsoft Graph PowerShell SDK and User.Read.All (used only to read the total source-user count for the coverage denominator). EXPECTED RESULTS A per-wave progress table, program totals (waves complete, users migrated vs source total, overall coverage %), and an optional CSV export. No source data is modified. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([Parameter(Mandatory)][string]$WavePlanCsv, [string]$CsvPath)

GWMIGPROG-Set-ScopeTargeting.ps1

Purpose: Creates (or reuses) the Microsoft Entra security group that defines a migration wave — the pilot (Wave 0) first, then each expansion wave — so every wave runs against a bounded, well-understood population and coexistence routing can key off it. Optionally seeds members from a CSV of user UPNs. This is the only included script that writes to the tenant: it creates a group and adds members, nothing else — no licenses changed, no mail flow altered, no data moved. Runs with -WhatIf by default. HOW TO RUN Preview (default): powershell -ExecutionPolicy Bypass -File .\GWMIGPROG-Set-ScopeTargeting.ps1 `-GroupName "GWMIGPROG Wave - Wave 1 - Finance EMEA" -MembersCsv .\wave1-users.csv Apply: ...\GWMIGPROG-Set-ScopeTargeting.ps1` -GroupName "GWMIGPROG Wave - Wave 1 - Finance EMEA" -MembersCsv .\wave1-users.csv ` -WhatIf:$false MembersCsv is a CSV with a 'Upn' column (users). Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All (and User.Read.All to resolve UPNs). EXPECTED RESULTS In preview: the group that would be created/reused and the members that would be added. When applied: the security group exists and contains the listed members, ready to target with the wave's cutover runbook. .NOTES White-label — the repeatable way. Writes to Microsoft Entra; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param(

[Parameter(Mandatory)][string]$GroupName, [string]$MembersCsv)

How to run: Preview (default): powershell -ExecutionPolicy Bypass -File .\GWMIGPROG-Set-ScopeTargeting.ps1 `-GroupName "GWMIGPROG Wave - Wave 1 - Finance EMEA" -MembersCsv .\wave1-users.csv` Apply: `...\GWMIGPROG-Set-ScopeTargeting.ps1` -GroupName "GWMIGPROG Wave - Wave 1 - Finance EMEA" -MembersCsv .\wave1-users.csv ` -WhatIf:$false MembersCsv is a CSV with a 'Upn' column (users). Requires the Microsoft Graph PowerShell SDK and Group.ReadWrite.All (and User.Read.All to resolve UPNs). EXPECTED RESULTS In preview: the group that would be created/reused and the members that would be added. When applied: the security group exists and contains the listed members, ready to target with the wave's cutover runbook. .NOTES White-label — the repeatable way. Writes to Microsoft Entra; -WhatIf by default. #> [CmdletBinding(SupportsShouldProcess = $true, ConfirmImpact = 'High')] param([Parameter(Mandatory)][string]$GroupName, [string]$MembersCsv)

GWMIGPROG-Validation.ps1

Purpose: Validates that a migration wave is ready to cut over: the wave's users (from a members CSV) exist in the Microsoft 365 / Entra source, are licensed, and the wave's target-group and coexistence signals are asserted. This is the readiness gate for the per-wave cutover runbook. Read-only against Microsoft Graph — it changes nothing. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\GWMIGPROG-Validation.ps1 -Wave "Wave 1" ` -MembersCsv .\wave1-users.csv -CoexistenceLive -TargetGroupReady -MembersCsv is a CSV with a 'Upn' column listing the wave's users. -CoexistenceLive asserts mail routing/calendar interop is live; -TargetGroupReady asserts the wave's Google target group is provisioned. Requires the Microsoft Graph PowerShell SDK and User.Read.All. EXPECTED RESULTS "WAVE READY" when every listed user resolves and is licensed and both coexistence and target-group signals are asserted; otherwise a per-signal GAP list and a non-zero exit code. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$Wave = 'Wave 0', [Parameter(Mandatory)][string]$MembersCsv, [switch]$CoexistenceLive, [switch]$TargetGroupReady)

How to run: powershell -ExecutionPolicy Bypass -File .\GWMIGPROG-Validation.ps1 -Wave "Wave 1" ` -MembersCsv .\wave1-users.csv -CoexistenceLive -TargetGroupReady -MembersCsv is a CSV with a 'Upn' column listing the wave's users. -CoexistenceLive asserts mail routing/calendar interop is live; -TargetGroupReady asserts the wave's Google target group is provisioned. Requires the Microsoft Graph PowerShell SDK and User.Read.All. EXPECTED RESULTS "WAVE READY" when every listed user resolves and is licensed and both coexistence and target-group signals are asserted; otherwise a per-signal GAP list and a non-zero exit code. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. #> [CmdletBinding()] param([string]$Wave = 'Wave 0', [Parameter(Mandatory)][string]$MembersCsv, [switch]$CoexistenceLive, [switch]$TargetGroupReady)